Mongo DB

MongoDB

- MongoDB and traditional RDBMS (Relational Database Management Systems) serve different use cases and have distinct advantages.
- Some key points highlighting why MongoDB might be considered better in certain scenarios.

- **Schema Flexibility:** MongoDB is schema-less, allowing you to store documents with varying structures. This is beneficial for applications that evolve quickly or require frequent changes to the data model.

- **Scalability:** MongoDB is designed for horizontal scaling through sharding, which allows it to distribute data across multiple servers easily. This makes it suitable for handling large volumes of data and high traffic.

- **Performance:** With its document-oriented structure, MongoDB can retrieve data faster for certain types of queries, especially when dealing with large datasets and complex data relationships.

MongoDB

- **Complex Data Types:** MongoDB supports rich data types, including arrays and nested documents, allowing for more complex data representations that can be more intuitive for developers.

- **Built-in Replication and High Availability:** MongoDB has robust features for replication and automatic failover, ensuring high availability and data redundancy without extensive setup.

- **Real-Time Analytics:** MongoDB can handle real-time analytics efficiently, making it a strong choice for applications that require immediate insights from their data.

MongoDB

- **RDBMS** is preferred for applications requiring strict data integrity and complex relationships, while **MongoDB** is chosen for applications needing flexibility, scalability, and the ability to handle varied data types.
- The choice ultimately depends on the specific requirements of the project, including performance, scalability, and data complexity.

MongoDB

- **Breakdown of the types of applications and developers that typically use RDBMS and MongoDB.**

- **Applications Typically Using RDBMS:**
  1. **Enterprise Applications**:
     1. **Developers**: Business application developers.
     2. **Use Case**: Applications with complex transactions, such as ERP, CRM, and financial systems, where data integrity and ACID compliance are crucial.
  2. **E-commerce Platforms**:
     1. **Developers**: E-commerce and web application developers.
     2. **Use Case**: Systems requiring robust transaction management, such as managing orders, inventory, and customer data.
  3. **Content Management Systems (CMS)**:
     1. **Developers**: Web developers and content strategists.
     2. **Use Case**: Applications that manage structured content with relationships between entities.

MongoDB

4. Banking and Financial Systems:

1. **Developers:** Fintech developers.
2. **Use Case:** Applications needing high security, data consistency, and complex queries.

5. Health Information Systems:

1. **Developers:** Health tech developers.
2. **Use Case:** Applications that manage sensitive patient data and require compliance with regulations.

MongoDB

- **Applications Typically Using MongoDB:**

1. **Web and Mobile Applications**:
    1. **Developers**: Full-stack developers and mobile app developers.
    2. **Use Case**: Applications with rapidly changing data requirements, such as social media platforms or messaging apps.

2. **Big Data and Analytics**:
    1. **Developers**: Data scientists and data engineers.
    2. **Use Case**: Applications that handle large volumes of unstructured data for real-time analytics.

3. **IoT Applications**:
    1. **Developers**: IoT developers.
    2. **Use Case**: Systems collecting and processing varied data from numerous devices, where flexibility in data structure is advantageous.

MongoDB

4. Content Management and Delivery:

1. **Developers**: Developers of media and content platforms.
2. **Use Case**: Applications that need to manage diverse content types and allow for quick iterations.

5. Gaming Applications:

1. **Developers**: Game developers.
2. **Use Case**: Games that require rapid updates and manage large amounts of user-generated content.

6. Real-time Data Applications:

1. **Developers**: Developers of real-time analytics and monitoring systems.
2. **Use Case**: Applications that need to process and respond to data in real-time, such as dashboards and event logging.

MongoDB

- MongoDB is an open-source document database and leading NoSQL database.
- MongoDB works on concept of collection and document.

- **Database :**
  - Database is a physical container for collections.
  - Each database gets its own set of files.
  - A single MongoDB server typically has multiple databases.

- **Collection :**
  - Collection is a group of MongoDB documents.
  - It is the equivalent of an RDBMS table.
  - A collection exists within a single database.
  - Collections do not enforce a schema.
  - Documents within a collection can have different fields.

MongoDB

- **Document :**
  - A document is a set of key-value pairs.
  - Documents have dynamic schema.
  - Dynamic schema means that documents in the same collection do not need to have the same set of fields or structure.

- The following table shows the relationship of RDBMS terminology with MongoDB.

RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row	Document
column	Field
Table Join	Embedded Documents
Primary Key	Primary Key (Default key **_id** provided by MongoDB itself)

SQL MySQL	vs	NoSQL MongoDB
• RDBMS is a relational database management system and works on relational database.		• It's a non-relational, document-oriented database management system and works on document-based database.
• It stores data in form of entity as tables. • It uses SQL to query database.		• MongoDB stores data in form of documents • MongoDB uses BSON to query database

Data formats

In MongoDB, it uses JSON and BSON formats.

JSON

JSON stands for JavaScript Object Notation. JSON syntax is based on a subset of JavaScript ECMA-262 3rd edition.

A JSON document is a collection of fields and values in a structured format. For example:

```
{
    "first_name": "John",
    "last_name": "Doe",
    "age": 22,
    "skills": ["Programming","Databases", "API"]
}
```

BSON

BSON stands for Binary JSON, which is a binary-coded serialization of JSON-like documents

Show all databases

- To show all available databases,
  - **show dbs**

- Change or Create a Database
  - **use databaseName**
- **Note** : An empty database is essentially non-existant. In MongoDB, a database is not actually created until it gets content.

- **Create Collection :** There are 2 ways to create a collection.

- **createCollection() :** Create a collection using the createCollection() database method.
  - **db.createCollection(“**collectionName**")**

- **Insert process :** Create a collection during the insert process.
  - **db.collectionName.insertOne(**object**)**
- This will create the collection if it does not already exist.

Insert Documents

- There are 2 methods to insert documents into a MongoDB database.

- **insertOne() :** To insert a single document, use the **insertOne()** method. This method inserts a single object into the database.

```
db.collection.insertOne(
    <document>
)
```

- **insertMany() :** To insert multiple documents at once, use the **insertMany()** method. This method inserts an array of objects into the database.

```
db.collection.insertMany(
    [document1, document2, ...]
)
```

Select Data

- There are 2 methods to find and select data from a MongoDB collection

- **find() :** To select data from a collection in MongoDB, we can use the find() method.
  - **db.Collection_name.find(selection_criteria, projection)**

- **selection_criteria:** It specifies selection criteria. To return all documents in a collection use empty document({}). The type of this parameter is document.
- **projection:** It specifies the fields to return in the documents that match the selection criteria. To return all fields in the matching documents, remove this parameter. It is of the document type.

- **findOne() :** To select only one document, we can use the findOne() method.This method accepts a query object. If left empty, it will return the first document it finds.
  - **db.collectionName.findOne()**

- **Note:** This method only returns the first match it finds.

Select Data

- **Querying Data :** To query or to filter, data we can include a query in our **find()** or **findOne()** methods.
  - **db.collectionName.find({field1:value , field2:value, … })**

- **Projection :** This parameter is an object that describes which fields to include in the results. Both find methods accept a second parameter called projection.
- This parameter is optional. If omitted, all fields will be included in the results.
  - **db.collectionName.find({}, {field1: 1, field2: 1})**
- **Use a 1 to include a field and 0 to exclude a field.**
- The **_id** field is also included. This field is always included unless specifically excluded.
  - **db.collectionName.find({}, { _id:0 , field1: 1, field2: 1})**

MongoDB Query Operators

- There are many query operators that can be used to compare and reference document fields.
- **Comparison :** The following operators can be used in queries to compare values:
  - **$eq:** Values are equal
  - **$ne:** Values are not equal
  - **$gt:** Value is greater than another value
  - **$gte:** Value is greater than or equal to another value
  - **$lt:** Value is less than another value
  - **$lte:** Value is less than or equal to another value
  - **$in:** Value is matched within an array

  - **Syntax:**

```
{ <field>: { $eq: <value> } }
```

```
{ field: { $in: [<value1>, <value2>,...] }}
```

MongoDB Query Operators

- **Logical**
- The following operators can logically compare multiple queries.
  - **$and:** Returns documents where both queries match
  - **$or:** Returns documents where either query matches
  - **$not:** Returns documents where the query does not match
  - **Syntax:**

```
$and :[{expression1}, {expression2},...]
```

```
db.employees.find(
    {
        age:{
            $gte:35
        }
    })
```

```
db.employees.find(
    {
        age:{
            $not:
            {
                $gte:35
            }
        }
    })
```

MongoDB $exists Operators

- The **$exists** is an element query operator that has the following syntax:

```
{ field: { $exists: <boolean_value> } }
```

- When the <boolean_value> is true, the $exists operator matches the documents that contain the field with any value including null.

- If the <boolean_value> is false, the $exists operator matches the documents that don't contain the field.

```
db.DemoCollection.find(
        {
            email:{$exists:true}
        }
    )
```

```
db.DemoCollection.find(
        {
            email:{$exists:true , $eq:'kohli@gmail.co'}
        }
    )
```

MongoDB $type Operators

- The $type is an element query operator that allows you to select documents where the value of a field is an instance of a specified BSON type.

- **The $type operator has the following syntax:**

```
{ field: { $type: <BSON type> } }
```

- MongoDB provides you with three ways to identify a BSON type: string, number, and alias.

MongoDB $type Operators

Type	Number	Alias
Double	1	“double”
String	2	“string”
Object	3	“object”
Array	4	“array”
Binary data	5	“binData”
ObjectId	7	“objectId”
Boolean	8	“bool”
Date	9	“date”
Null	10	“null”

MongoDB $type Operators

```
db.DemoCollection.find(
        {
            email:{$type:"string" ,$exists:true , $eq:'kohli@gmail.co'}
        }
    )
```

- Select documents where the price field is either number or string or an array that has an element is number or string:

```
db.DemoCollection.find(
        {
            age:{$type:["number","string"] ,$exists:true , $gte:45}
        }
    )
```

MongoDB Update Document

- To update an existing document MongoDB uses **updateOne()** or **updateMany()** methods.
- The first parameter is a query object to define which document or documents should be updated.
- The second parameter is an object defining the updated data.

- **updateOne() :** The **updateOne**() method will update the first document that is found matching the provided query.

```
db.collection.updateOne(filter, update, options)
```

- **Insert if not found :** If you would like to insert the document if it is not found, you can use the **upsert** option.
  - **Syntax : db.collection.updateOne({ field1:value1 }, { $set: {field1: value1 }, ... }, { upsert: true })**

- **updateMany() :** The updateMany() method will update all documents that match the provided query.

MongoDB Update Operators

- There are many update operators that can be used during document updates.

- The following operators can be used to update fields:

  - **$set:** Sets the value of a field.
    - **Syntax : {$set: { <field1>: <value1>, <field2>: <value2>, ... } }**
    - **Eg : db.Collection1.updateOne({unm:“John"},{$set:{age:22}})**

  - **$rename:** Renames the field
    - **Syntax : {$rename: { <field1>: <newName1>, <field2>: <newName2>, ... } }**
    - **Eg : db.Collection1.updateMany({},{$rename:{"nm":"unm"}})**

  - **$unset:** Removes the field from the document.
    - **Syntax : { $unset: { <field1>: "", ... } }**
    - **Eg : db.Collection1.updateOne({unm:“John"},{$unset:{age:""}})**

MongoDB Delete Documents

- Delete documents by using the methods **deleteOne()** or **deleteMany().**
- These methods accept a query object. The matching documents will be deleted.

- **deleteOne() :** The deleteOne() method will delete the first document that matches the query provided.

- **deleteMany() :** The deleteMany() method will delete all documents that match the query provided.

MongoDB limit() Method

- The **limit()** method in MongoDB is used to specify the maximum number of documents that should be returned in a query result.
- It is particularly useful for controlling the size of the result set especially when dealing with large collections.
- By using **limit()**, which can improve performance and reduce the load on the client.

- **Syntax : db.Collection_name.find(selection_criteria, projection).limit(n)**

MongoDB skip() Method

- **skip()** method is used to skip a specified number of documents in a query result.
- This allows us to control the starting point of the returned results which effectively removes the first **n** documents from the result set.

- **Syntax : db.Collection_name.find(selection_criteria, projection).skip(n)**

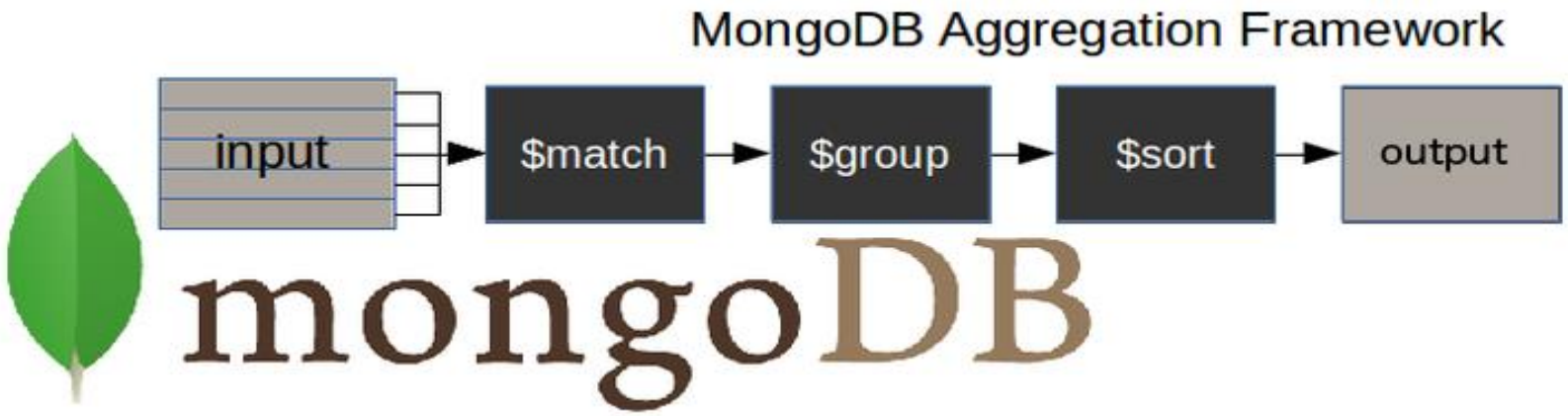
MongoDB sort() Method

- The **sort()** method in MongoDB is used to specify the order in which the query returns matching documents from a given collection.
- It takes a document as a parameter that contains a field: value pair that defines the sort order of the result set.
- The value is 1 or -1 specifying an ascending or descending sort respectively.

- **Syntax : db.Collection_name.find(selection_criteria, projection). sort({ field1: 1 or -1})**

Aggregation Pipeline

- Aggregation operations allow you to group, sort, perform calculations, analyze data, and much more.
- Aggregation pipelines can have one or more "stages". The order of these stages are important. Each stage acts upon the results of the previous stage.
- MongoDB's aggregation framework is modeled on the concept of data processing pipelines. Documents enter a multi-stage pipeline that transforms the documents into an aggregated result. For example:



- $match() stage – filters those documents we need to work with, those that fit our needs
- $group() stage – groups documents
- $sort() stage – sorts the resulting documents the way we require (ascending or descending)

```
db.collection.aggregate([{ $match:...},{$group:...},{$sort:...}]);
```

- Call the aggregate() method on the collection.
- Pass an array of documents, where each document describes a stage in the pipeline.

MongoData.txt

MongoDB $project

- The $project is used to return only those fields you need so as to avoid processing more data than is necessary.
- In this example, we only need the fields country,city and name.
- In the code that follows, please note that:
  - We must explicitly write _id:0 when this field is not required.

  - db.universities.aggregate([{$project:{_id:0,country:1,city:1,name:1}}]).pretty()

MongoDB $match

- The $match stage allows us to choose just those documents from a collection that we want to work with.
- It does this by filtering out those that do not follow our requirements.

```
db.employees.aggregate([ {$match:{ gender: 'female'}} ])
```

- In the above example, the $match stage is specified as a document {$match:{ gender: 'female'}} in an array.
- It will return all documents where gender:'female' filed.

MongoDB $group

- $group stage to group the input documents by the specified _id expression and returns a single document containing the accumulated values for each distinct group

```
db.employees.aggregate([
    {
        $group:{ _id:'$department.name'}
    }
])
```

- **_id:** This field acts as the unique identifier for each group that will be created.
- **$department.name**: This references the name field inside the department sub-document.

MongoDB $group

- The expression { _id:'$department.name'} creates the distinct group on the field $department.name.

- Since we don't calculate any accumulated values, it returns the distinct values of $department.name

- **Note :** Accumulators are operators that maintain their state (e.g. totals, maximums, minimums, and related data) as documents progress through the pipeline.

Aggregation Pipeline Operators

- **Arithmetic Expression Operators**
- It is used to perform arithmetic operations on numbers. Some arithmetic expression also supports data arithmetic.
- **$sum**

```
db.employees.aggregate([
    {
        $group : { _id : '$department.name' , TotalSalary : {$sum: '$salary'} }
    }

])
```

- **$avg**

```
db.employees.aggregate([
    {
        $group : { _id : '$department.name' , AverageSalary : {$avg: '$salary'} }
    }
])
```

- **$min**

```
db.employees.aggregate([
    {
        $group:{ _id:'$department.name',MinimumSalary:{$min:'$salary'}}
    }
])
```

- **$max**

```
db.employees.aggregate([
    {
        $group:{ _id:'$department.name',MaximumSalary:{$max:'$salary'}}
    }
])
```

MongoDB $sort

- The $sort stage is used to sort the documents based on the specified field in ascending or descending order.

```
db.employees.aggregate([
    { $match:{ gender:'male'}},
    { $sort:{ firstName:1}}
])
```

- The { $sort:{ firstName:1}} expression sorts the input documents by the firstName field in ascending order. 1 indicates the ascending order and -1 indicates descending order.

MongoDB $sort

```
db.employees.aggregate([
    { $match:{ gender:'male'}},
    { $group:{ _id:{ deptName:'$department.name'}, totalEmployees: { $sum:1} } },
    { $sort:{ totalEmployees:1}}
])
```

- The { $sort:{ totalEmployees:1}} expression sorts the input documents by the totalEmployees field in ascending order. 1 indicates the ascending order and -1 indicates descending order.

MongoDB $limit

- This aggregation stage limits the number of documents passed to the next stage.

```
db.employees.aggregate([
    { $match:{ gender:'male'}},
    { $sort:{ firstName:1}},
    {$limit:3}
])
```

This will return the 3 document from the collection.